Kotlin Backend Developer.Professional

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Разработка приложения со Spring

otus.ru

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Поздравляю!
Это экватор (21 занятие из 41)

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Маркетинг аналитика	Security	Архитек-тура	DevOps		
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Консуль-тация	Защита				

Команда курса

Окатов Сергей
автор программы и руководитель курса

- Кандидат физико-математических наук,
- 15 лет научной работы в университетах России, Европы, США;
- Руководитель отдела разработки БФТ.ЦР;
- Работа в стартапах и энтерпрайзе, BigData, Backend, Frontend.

Кирилл Крылов
преподаватель

- Kotlin/ML Разработчик в JetBrains;
- Веду курсы Android-разработки в Samsung;
- Full-stack разработчик мобильных приложений на Kotlin/TS;
- Руководитель курса Kotlin Basic.

Земенков Леонид
преподватель

- Kotlin разработчик в Ennabl, Inc.;
- 4 года пишу на Java;
- За 4 года успел поработать в Сбербанке, Yandex и AliExexpress.

Непомнящий Евгений
преподаватель

- 15 лет программировал контроллеры на C++; и руководил отделом разработки
- 3 года пишу на Java;
- Последнее время пишу микросервисы на Java в M-Tex.

Таношкин Александр
преподаватель

- Инженер-программист в крупном классифайде недвижимости;
- Работал с Java, TS и другими языками;
- На Kotlin с 2019 г.

Порхун Анастасия
преподаватель

- 10 лет в индустрии
- Опыт работы в телеком- и аутсорскомпаниях, с гос. структурами;
- Проектирование и разработка ПО;
- Выстраивание процессов CI/CD, исследования в сфере ИБ.

Миних Владимир
преподаватель

- 1991 г. – моя первая программа (ZX Spectrum, Basic);
- 1998 г. – Delphi + первый Java опыт;
- 15+ лет – инженер АСУ ТП. Разработка ПО для ПЛК;
- с 2014 г. – Java разработчик, с 2018 г. – опыт с Kotlin.

Никольский Николай
преподаватель

- Java/Kotlin разработчик в ОТП банке.
- Работа с крупными государственными и банковскими проектами.
- Участник opensource

OTUS | ОНЛАЙН ОБРАЗОВАНИЕ

План полета

- Особенности Spring
- Разработка приложения
- Интеграция с транспортом
- Деплой в Docker
- Рефлексия

Цели вебинара

После занятия вы сможете

1. Создавать Spring приложения
2. Интегрировать с транспортными моделями
3. Выполнять упаковку в Docker

Смысл

Зачем вам это уметь

1. Spring - самый распространенный JVM-фреймворк
2. Интеграция с транспортными моделями позволяет максимально быстро развернуть заглушку приложения
3. Запуск в виде контейнеров - основной способ боевого развертывания

О Spring

Spring Framework

- Spring – универсальный фреймворк
- Первая публикация в 2002 году
- Большой набор функций и интеграций
  - Spring JDBC
  - Spring MVC
  - Spring Security
  - Spring AOP
  - Spring ORM
  - Spring Test
  - Spring Cloud
  - etc

Spring Boot

- Spring Boot – расширение Spring для облегчения конфигурирования приложения
- Предоставляет опциональный 'starter' POMs для упрощения Maven конфигурации
- Автоматически конфигурирует Spring где это возможно
- Предоставляет production-ready features такие как метрики, самодиагностика (health check) и внешняя конфигурация (Actuator)
- Никакой кодогенерации и никаких XML-конфигураций

О Spring core

Spring

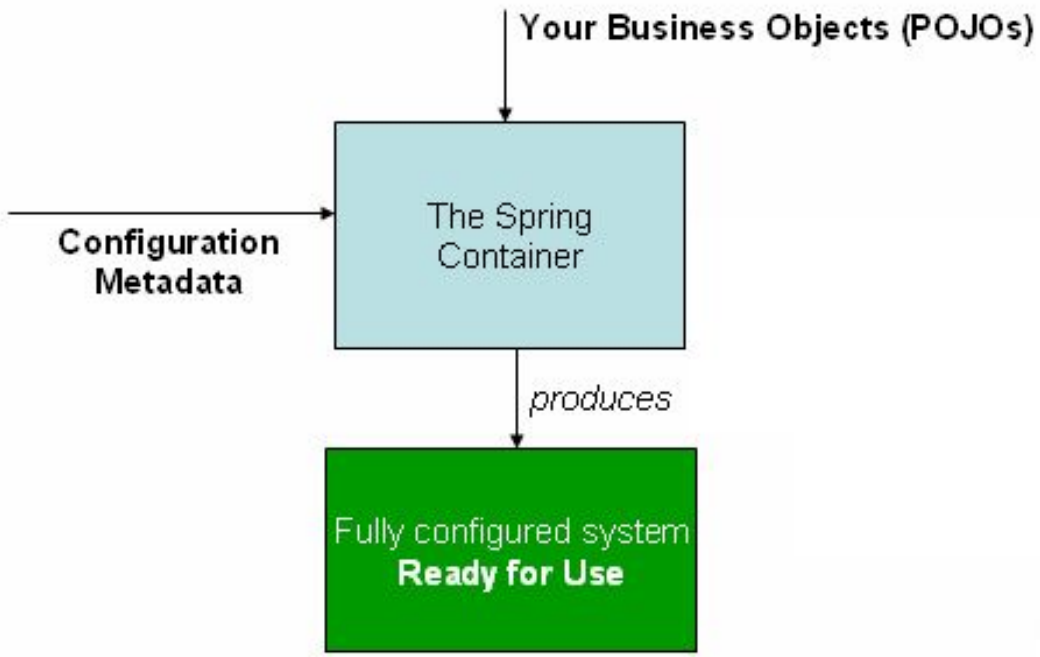
Общие сведения:

1. Основатель Rod Johnson
2. Создан в 2002 году
   (Java 8 только в 2014)
3. Прорывные концепции:
   a. IoC – inversion of control (Spring Beans)
   b. AOP – aspect oriented programming

Spring IoC

IoC:

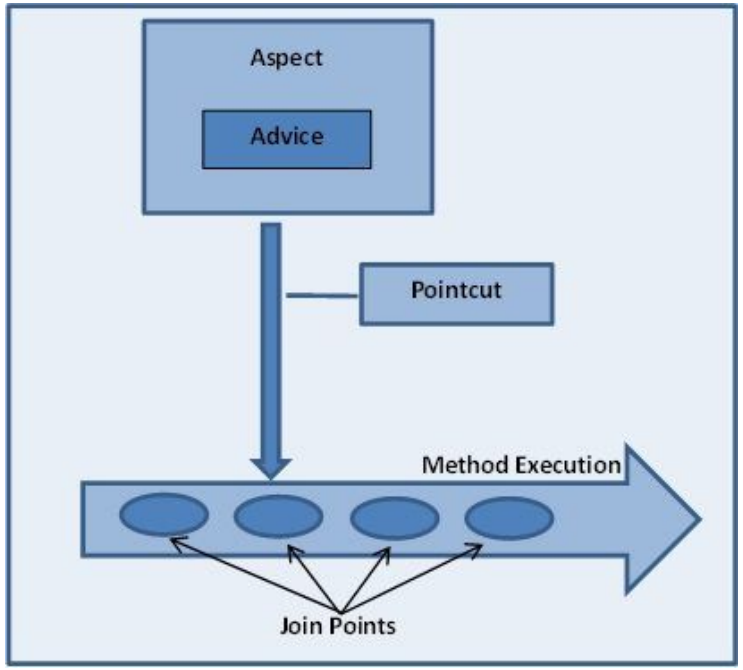
1. Реализован через интерфейсы BeanFactory и ApplicationContext
2. Выполняет настройку и связывание компонентов приложения на этапе инициализации
3. Прячем всю логику инициализации и связи между объектами



Spring AOP

AOP:

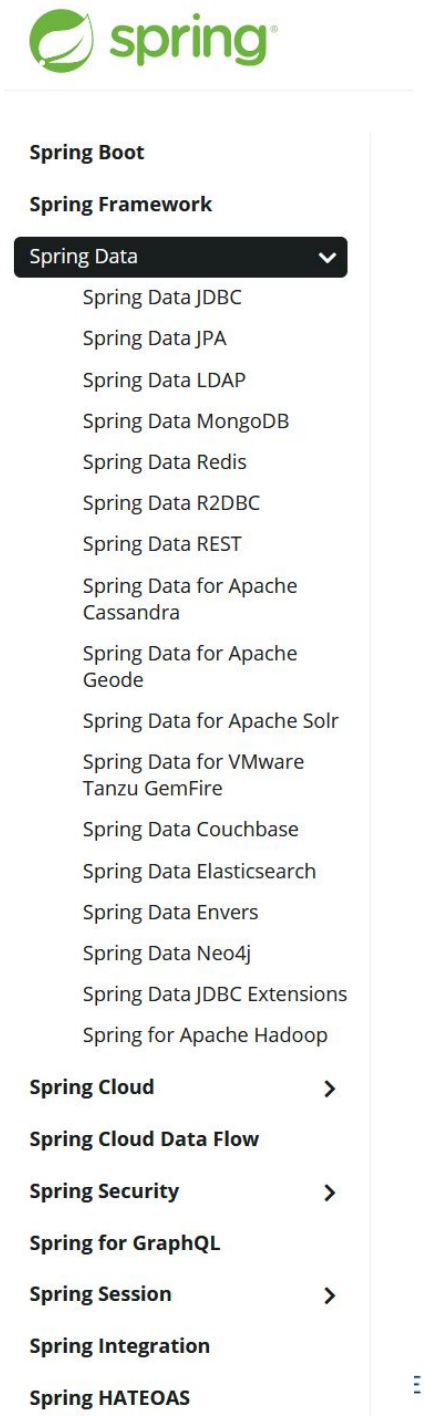
1. Отделение сервисной логики от бизнес-логик (логирование, транзакции)
2. Примеры: @Test, @PostMapping, @Transactional
3. AOP Proxies
    a. JDK dynamic proxies - только через интерфейсы
    b. CGLIB – через наследование - all open
    c. @AspectJ support - мощная библиотека
4. Создание класса в 4-10 раз медленнее



Spring сегодня

“Зонтик” для большого числа разных проектов. Обычно проект представляет собой обертку над какой-то сторонней библиотекой/фреймворком и позволяет удобно использовать ее из spring-приложения

https://spring.io/projects



Spring boot

Spring Boot

Выполняет автоконфигурацию на основе “разумных умолчаний” исходя из подключенных зависимостей.

https://spring.io/projects/spring-boot

https://docs.spring.io/spring-boot/docs/2.7.2/reference/htmlsingle/

Можно вмешаться и подстроить те моменты, которые вам нужны - через свойства в application.yaml, через параметры командной строки, через свои собственные бины.

Из коробки предоставляет много всего нужного для микросервисов - логирование, трейсинг, метрики, health-check и т.п.

Критика

1. Концепция AOP морально устарела и не имеет преимуществ перед DSL/lambda
2. Подход Spring MVC недостаточно разделяет код, чем навязывает высокую связанность
3. Spring JPA размазывает работу с БД по всему проекту, повышая связность кода
4. Любой проект на Spring чаще всего превращается в спагетти
5. Производительность Spring-приложений уступает многим другим фреймворкам

https://www.techempower.com/benchmarks/#section=data-r22&hw=ph&test=composite

49	es4x	853,365	197,140	11,744	102,788	8,339	3,029,481	**2,934**	36.3%
50	actframework	964,004	231,641	16,942	124,422	1,985	3,273,101	**2,911**	36.0%
51	ratpack	478,204	238,918	27,307	118,257	11,109	803,281	**2,859**	35.4%
52	ktor	426,795	185,734	26,927	80,906	14,735	803,388	**2,803**	34.7%
53	openswoole	720,865	254,803	14,442	201,297	4,868	871,531	**2,773**	34.3%
82	rack	384,609	82,662	11,391	56,590	7,316	574,556	**1,600**	19.8%
83	akka-http	198,321	138,856	10,421	89,288	1,913	2,635,450	**1,590**	19.7%
84	clevergo	423,839	179,109	9,301	80,269	2,507	681,884	**1,546**	19.1%
85	spliffy	493,992	91,124	10,872	72,440	2,836	1,001,557	**1,541**	19.1%
86	fastapi	366,204	72,724	11,373	46,896	7,610	448,130	**1,524**	18.8%
87	treefrog	430,192	119,190	6,970	91,863	2,999	954,904	**1,523**	18.8%
88	spring	236,259	147,907	15,932	24,082	7,131	506,087	**1,507**	18.6%
89	saphir	687,786	54,094	2,872	54,685	2,731	1,513,231	**1,500**	18.6%
90	silicon	395,595	112,924	12,023	116,869	1,979	123,416	**1,485**	18.4%

Kotlin Spring

Классический Spring:

1. По умолчанию классы Kotlin final
2. kotlin("plugin.spring") - плагин для раскрытия классов

Kotlin Spring DSL:

1. Spring-Fu (experimental)
2. https://github.com/spring-projects-experimental/spring-fu/tree/master/kofu
3. Конфигурация через dsl, а не рефлексию
4. Быстрый старт

Kotlin Spring fu

```
val app = webApplication {
    beans {
        bean<SampleService>()
        bean<SampleHandler>()
    }
    webMvc {
        port = if (profiles.contains("test")) 8181 else 8080
        router {
            val handler = ref<SampleHandler>()
            GET("/", handler::hello)
            GET("/api", handler::json)
        }
        converters {
            string()
            jackson()
        }
    }
}

fun main() {
    app.run()
}
```

Резюме

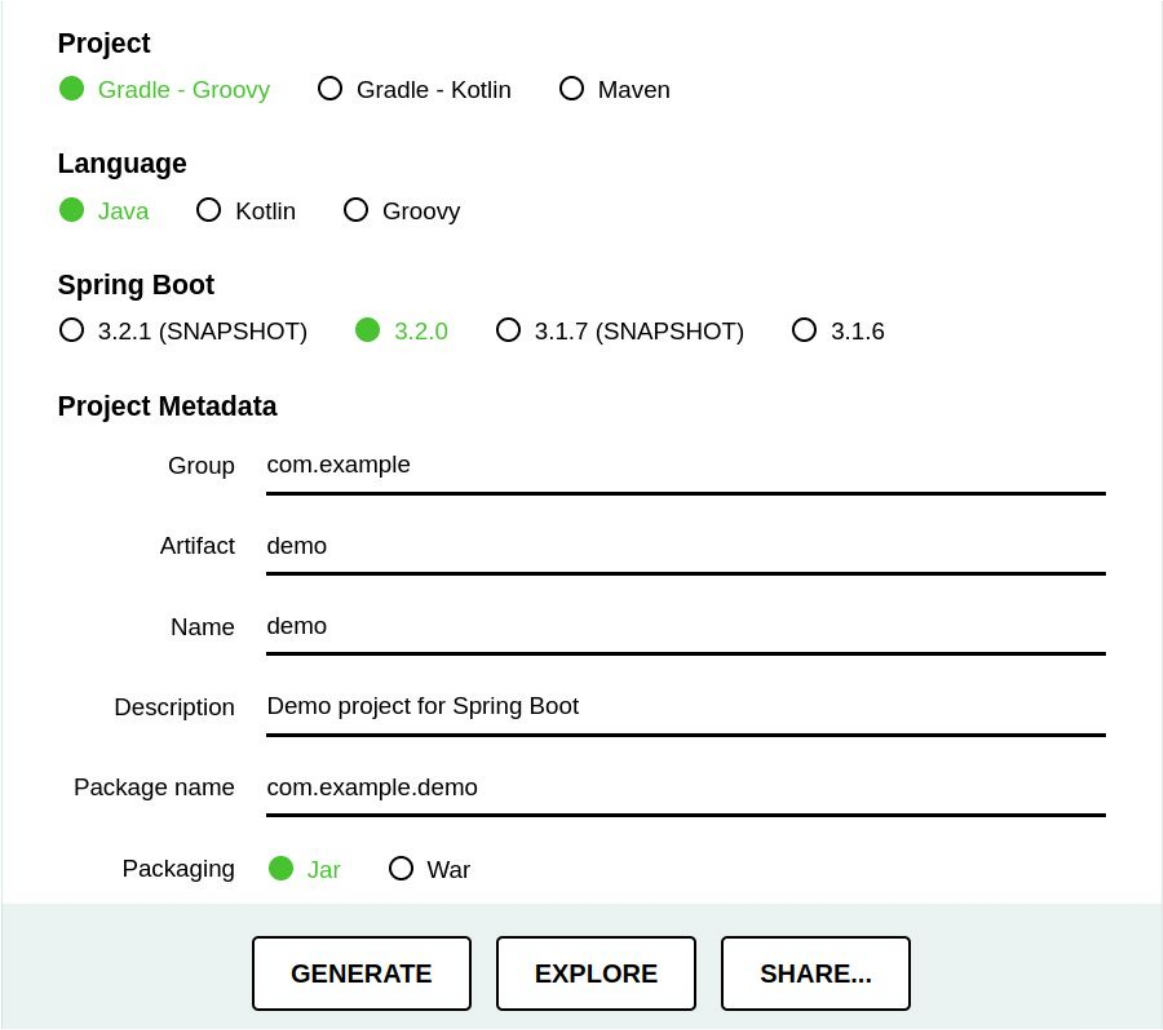
Spring:

1. Наиболее часто используемый фреймворк для бэкэнда
2. Kotlin Spring DSL - если хочется экзотики
3. JAVA framework: Null-safety все так же нет
4. Есть поддержка Kotlin - например корутины в r2dbc и webFlux

Как создать проект

Как создать проект

- https://start.spring.io/
- шаблон в Intellij Idea Ultimate
- вручную



Элементы Spring

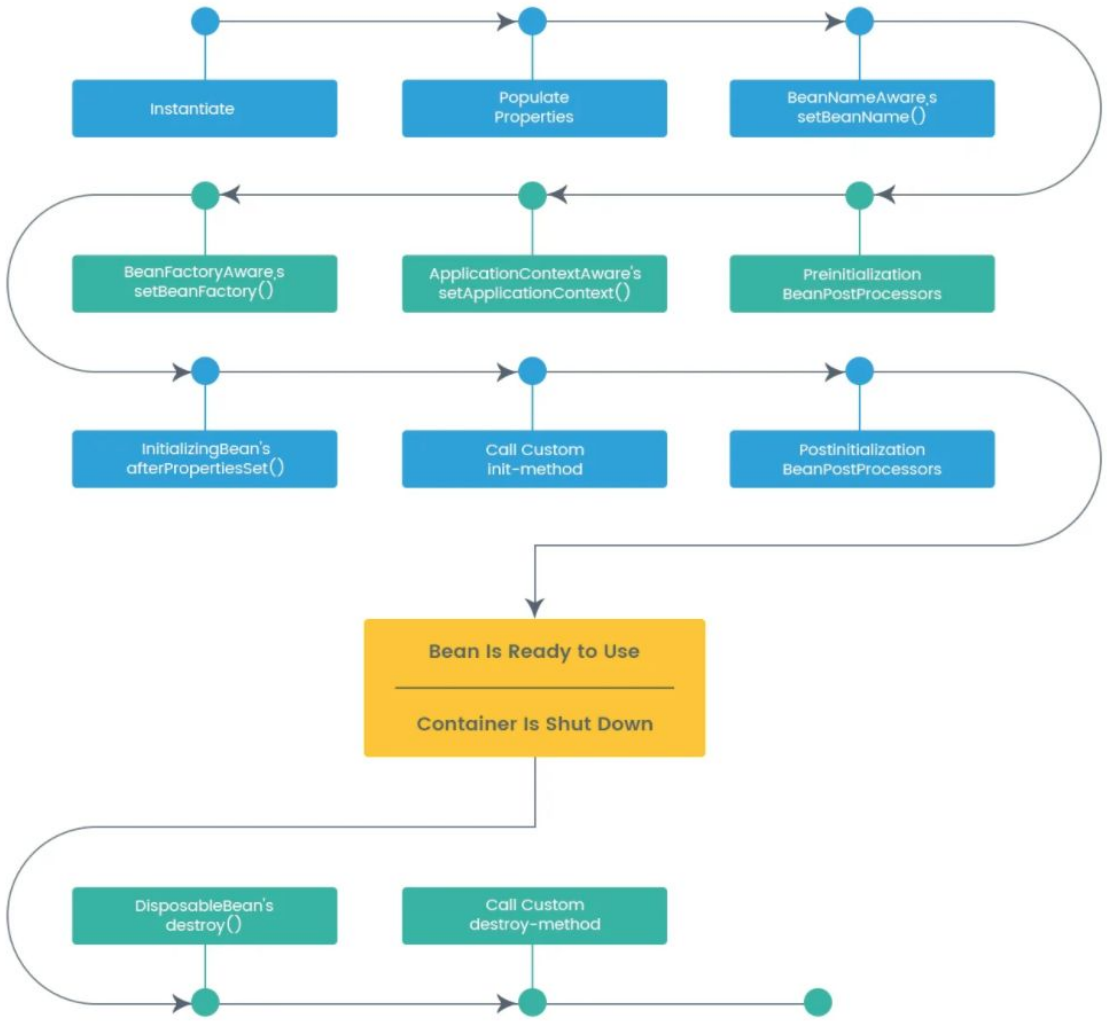
Bean

- Бин - это объект, который создан IoC контейнером
- IoC контейнер - это фабрика, которая создает бины
- IoC контейнер нам дает Spring, а бины пишем мы (или используем уже готовые)
- Бином может быть все что угодно (нет требований), но есть затраты на создание
- Обычно бином делают что-то долгоживущее (singleton), но есть скоупы: *singleton, prototype, request, session, application, websocket*

Доступ к бинам:

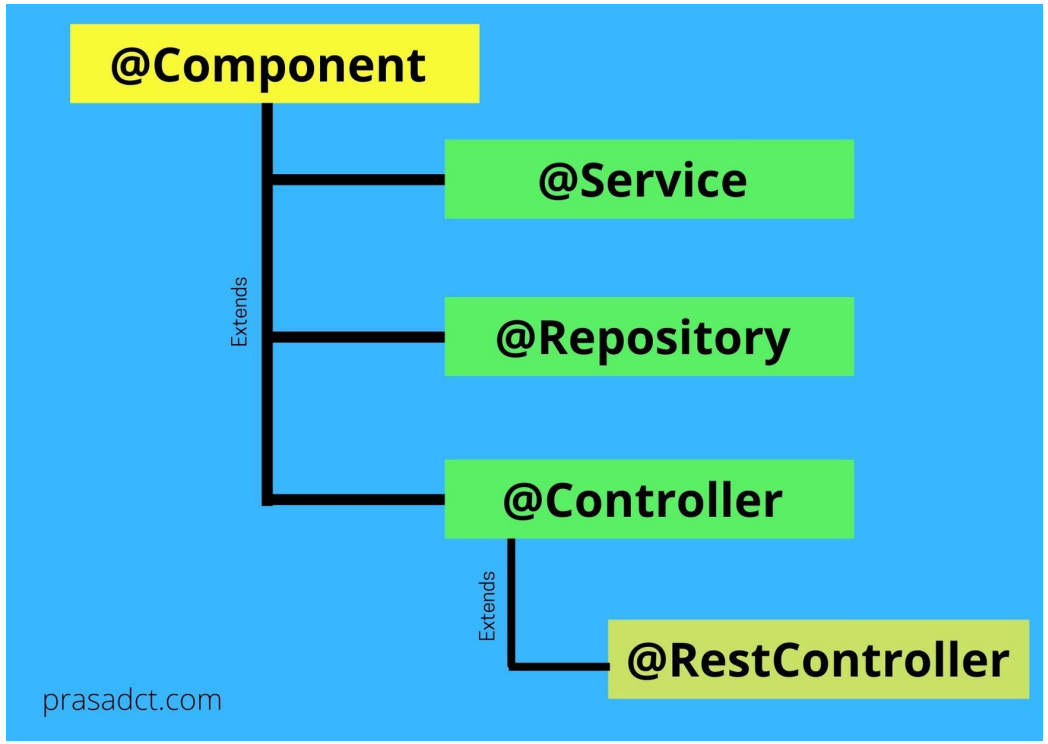
- @Autowire (не рекомендуется)
- Конструктор компонента

Жизненный цикл бина



Компоненты

- Виды:
  - @Component
  - @Service
  - @Repository
  - @Controller
    - @RestController



Конфигурации

- Виды конфигураций
  - @Value одиночные свойства из настроек (не рекомендуется)
  - @ConfigurationPropreties куски настроек (рекомендуется как ООП подход)
  - @Configuration - для бинов

Может потребоваться включение

@EnableConfigurationProperties

```
api:
    clientId: "client123"
    url: "https://api.url.com"
    key: "api-access-key"
```

```
@ConfigurationProperties(prefix = "api")
data class ApiConfiguration(
    var clientId: String = "",
    var url: String = "",
    var key: String = ""
)
```

```
@ConfigurationProperties(prefix = "api")
@ConstructorBinding
data class ApiConfiguration(
    val clientId: String,
    val url: String,
    val key: String
)
```

Главный класс - XxxApplication

- «запускатель» приложения runApplication<>()
- аннотация @SpringBootApplication

Запускатель

- создает контекст
- обрабатывает аргументы командной строки
- обновляет контекст

@SpringBootApplication

- @SpringBootConfiguration
  - @Configuration
- @EnableAutoConfiguration
- @ComponentScan

build.gradle.kts (Items2) | DemoApplication.kt | index.html

```kotlin
7
8  @SpringBootApplication
9  class DemoApplication
10
11 fun main(args: Array<String>) {
12     runApplication<DemoApplication>(*args)
13 }
14
15 @RestController
16 class MessageResource {
17     @GetMapping
18     fun index(): List<Message> = listOf(
19         Message( id: "1",  text: "Hello!"),
20         Message( id: "2",  text: "Bonjour!"),
21         Message( id: "3",  text: "Privet!"),
22     )
23 }
24
25 data class Message(val id: String?, val text: String)
```

Docker*

Зачем нужен Docker?

Docker - технология для создания и управления контейнерами.

Контейнер (container) содержит в себе все необходимое для запуска и работы приложения

- исполняемый код приложения
- зависимости приложения
- зависимости ОС
- библиотеки

Dockerfile: Java Hello World

```
FROM bellsoft/liberica-openjdk-alpine:17

COPY build/libs/ok-marketplace-app-spring-0.0.1.jar /app/



ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-jar",
"/app/ok-marketplace-app-spring-0.0.1.jar"]
```

```
$> docker build -t ok-marketplace-app-spring:v1 .

$> docker run --name marketplace -p 8080:8080 ok-marketplace-app-spring:v1
```

Так делать не надо: есть готовый плагин для генерации образов

LIVE

Рефлексия

Слайд с вопросами

1.	Какие прорывные технологии Spring?
2.	Какие особенности создания Spring-приложения на Kotlin?
3.	Какие бины нужны для создания Spring MVC/WebFlux приложения

5 мин

Слайд с тезисами

1.	Spring был прорывом 20 лет назад. Сейчас морально устаревший и медленный
2.	Прорывные технологии в нем - IoC и AOP
3.	Реализация через рефлексию, очень медленно, но часто не имеет большого значения
4.	kotlin(“spring.plugin”) для раскрытия final классов в классическом Spring
5.	Spring-Fu – экспериментальный функциональный стиль Spring

Слайд с домашним заданием

1.	Выбрать фреймворк для выпускного проекта и реализовать приложение на нем
2.	Подключить транспортные модели к приложению
3.	Реализовать автоматические тесты

Срок: 09.09.2025

Рефлексия

С какими основными мыслями и инсайтами уходите с вебинара?

Понимаете ли вы, как применять на практике то, что узнали на вебинаре. Если да, то как?

Цели вебинара

проверка достижения целей

1. Создавать Spring приложения
2. Интегрировать с транспортными моделями
3. Выполнять упаковку в Docker

Следующий вебинар

22.08.2025

Модуль 5: Фреймворки и асинхронный транспорт

Разработка приложения с KTOR

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

- https://github.com/otuskotlin/202312-otuskotlin-marketplace
- https://spring.io/
- https://start.spring.io/
- Spring boot + Docker: https://spring.io/guides/topicals/spring-boot-docker
- Spring-Fu: https://github.com/spring-projects-experimental/spring-fu
- Spring потрошитель: https://www.youtube.com/watch?v=BmBr5diz8WA
- https://habr.com/ru/company/otus/blog/679722/
- Плагин Spring: https://docs.spring.io/spring-boot/docs/current/gradle-plugin/reference/htmlsingle/

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары